Procesamiento Masivo de Datos

Hadoop

HDFS, YARN y MapReduce

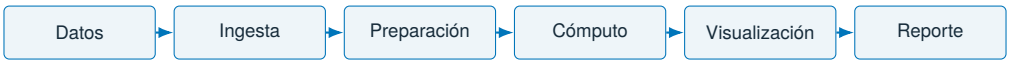
Alex Di Genova

Universidad de O'Higgins

De datos a información

Un flujo de trabajo de ciencia de datos

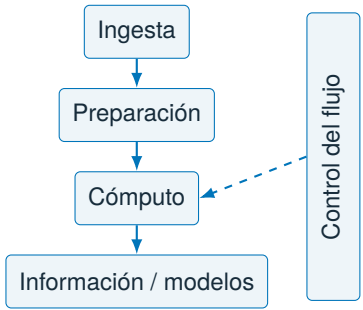
- Los grandes volúmenes pueden superar la memoria, el almacenamiento o el tiempo disponible en una sola máquina.
- El análisis masivo combina software paralelo con varios servidores.
- El flujo completo incluye preparar, procesar y comunicar los resultados.



Flujos de trabajo de Big Data

Escalables, reproducibles y automatizables

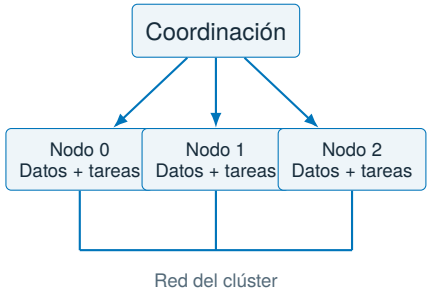
1. **Consumo:** localizar, incorporar y anotar datos.
2. **Preparación (staging):** transformar, normalizar y almacenar.
3. **Cómputo:** generar reportes y modelos de recomendación, agrupamiento o clasificación.
4. **Control:** orquestar tareas mediante scripts, aplicaciones y trabajos.



Hadoop: una plataforma para Big Data

Almacenamiento y procesamiento en un clúster

- **HDFS** almacena los archivos en varios nodos de cómputo.
- **MapReduce** divide la aplicación en tareas que pueden ejecutarse en paralelo y reintentarse.
- Los nodos procesan sus porciones de datos y combinan resultados.
- La plataforma gestiona fallas y coordina recursos.



un sistema operativo para Big Data

Hadoop ofrece servicios sobre el sistema operativo de cada máquina para construir aplicaciones distribuidas.

Requisitos de un sistema distribuido

Continuar operando y crecer con la carga

Requisito	Propósito
Tolerancia a fallas	Una falla de un componente puede atenderse sin detener todo el sistema.
Recuperación	Recuperar datos y trabajo mediante réplicas y reintentos.
Consistencia	Mantener resultados coherentes al recuperar una tarea fallida.
Escalabilidad	Incorporar recursos para admitir más datos o cómputo.

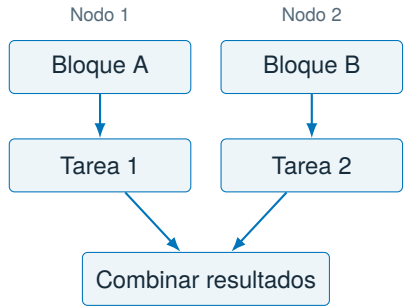
Capacidad y rendimiento

Agregar nodos puede aumentar la capacidad; la mejora depende del reparto del trabajo, la red y los costos de coordinación.

Acercar el cómputo a los datos

Localidad, tareas y recuperación

- Los archivos se dividen en bloques y se almacenan con réplicas.
- Un trabajo se divide en tareas; se busca ejecutarlas cerca de sus datos.
- El framework asigna trabajo y vuelve a ejecutar tareas fallidas.
- El programador describe el cálculo a alto nivel y reduce la coordinación explícita entre tareas.

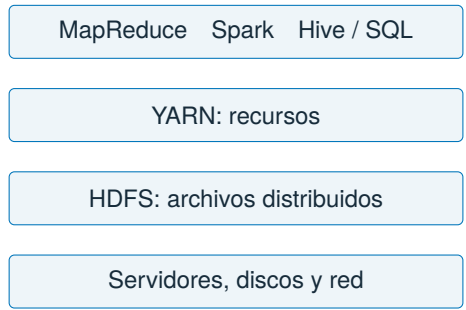


El shuffle puede transferir datos por la red.

Arquitectura de Hadoop

HDFS almacena; YARN administra recursos

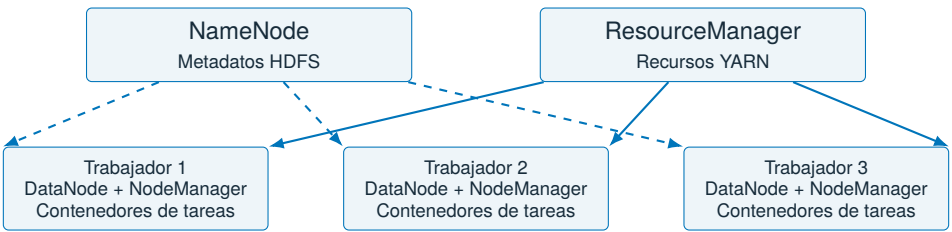
- **HDFS:** sistema de archivos distribuido sobre los discos del clúster.
- **YARN:** asigna CPU y memoria a las aplicaciones.
- **MapReduce:** motor de procesamiento que utiliza esos servicios.
- Otras aplicaciones pueden compartir la plataforma.



Un clúster Hadoop

Servicios de coordinación y nodos trabajadores

- Los servicios son procesos en segundo plano (demonios), comunicados por red.
- Los demonios Java se ejecutan en sus propias JVM.
- Los nodos de coordinación administran metadatos y recursos; los trabajadores almacenan datos y ejecutan tareas.



Esquema simplificado: los servicios pueden distribuirse en distintas máquinas.

Servicios HDFS

Metadatos separados de los bloques de datos

NameNode

Mantiene el espacio de nombres: directorios, archivos, metadatos y correspondencia entre bloques y DataNodes.

Secondary NameNode

Genera puntos de control que combinan la imagen del sistema de archivos y el registro de cambios.

DataNode

Almacena bloques en el sistema de archivos local y atiende lecturas y escrituras.

El Secondary NameNode no reemplaza automáticamente a un NameNode fallido; la alta disponibilidad utiliza otra configuración.

Servicios YARN

Recursos globales, coordinación por aplicación y ejecución local

ResourceManager

Administra la asignación de recursos del clúster entre aplicaciones.

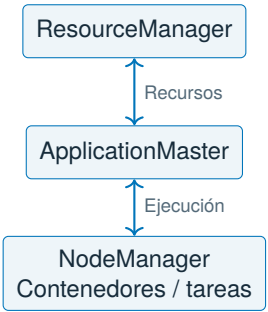
ApplicationMaster

Negocia recursos y coordina las tareas de una aplicación particular.

NodeManager

Administra los contenedores de un nodo y comunica su uso de recursos.

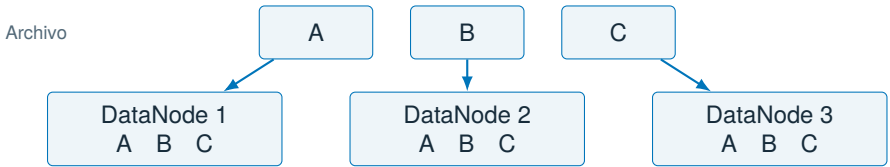
El ApplicationMaster se ejecuta en un contenedor del clúster, por aplicación.



HDFS: bloques y réplicas

Distribuir archivos grandes sobre discos locales

- HDFS usa los sistemas de archivos locales de los DataNodes, como ext4 o XFS.
- Los bloques son grandes y configurables; 128 MB es un tamaño habitual.
- La replicación mantiene varias copias de cada bloque; el ejemplo usa tres.



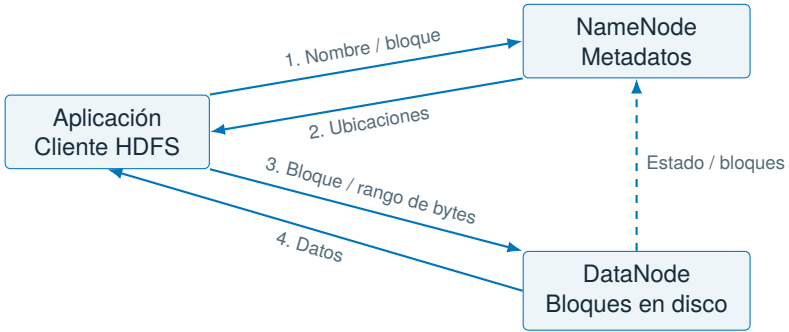
Redundancia

Tres réplicas consumen aproximadamente tres veces el espacio de los datos. Cada bloque sigue disponible mientras exista una réplica accesible.

El último bloque puede ser más pequeño; el bloque no es una unidad mínima de lectura.

Cómo se lee un archivo HDFS

El NameNode devuelve ubicaciones; el DataNode entrega los datos



Separar control y transferencia

Los datos del archivo viajan directamente entre el cliente y el DataNode.

Operaciones sobre HDFS

La consola de archivos: hadoop fs

```
hadoop fs -help
hadoop fs -mkdir -p text_db
hadoop fs -copyFromLocal kmers.txt text_db/
hadoop fs -ls text_db
hadoop fs -cat text_db/kmers.txt | less
hadoop fs -tail text_db/kmers.txt
```

```
hadoop fs -get text_db/kmers.txt
    ./copia.txt
hadoop fs -chmod 664
    text_db/kmers.txt
hadoop fs -test -e
    text_db/kmers.txt
hadoop fs -count text_db
hadoop fs -du -h text_db
```

Local y remoto

`copyFromLocal` sube archivos; `get` los descarga. `test -e` devuelve un estado de salida, sin imprimir el resultado.

Las rutas relativas se resuelven en el directorio del usuario del sistema de archivos configurado.

MapReduce: funciones sobre pares clave/valor

Tareas independientes y combinación de resultados

- **Map** transforma cada registro en cero o más pares intermedios.
- **Reduce** procesa los valores asociados a una misma clave.
- El modelo favorece funciones que dependen de sus entradas y evitan estado compartido entre tareas.

```
def map(key, value):
    # Emit zero or more pairs
    emit(intermediate_key,
         intermediate_value)
```

```
def reduce(key, values):
    # Aggregate values for this key
    emit(key, result)
```

Pseudocódigo: emit representa la emisión de un par.

La clave coordina el cálculo

Todos los valores de una clave deben llegar al mismo reductor.

Las fases de MapReduce

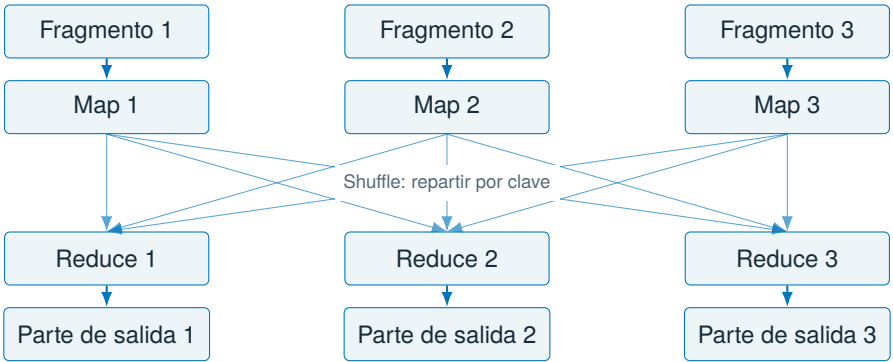
Entrada, map, shuffle y sort, reduce, salida



1. Leer registros desde la entrada, por ejemplo archivos en HDFS.
2. Emitir los pares clave/valor calculados por los mapeadores.
3. Particionar, transferir y ordenar los pares para agruparlos por clave.
4. Agregar cada grupo y escribir los pares finales.

Un trabajo distribuido

Los mapeadores leen fragmentos; el shuffle conecta con los reductores



Un reductor puede recibir datos de todos los mapeadores. La salida se reparte en archivos.

Ejemplo: contar apariciones de números

El mapper emite una unidad por registro

`input.txt`

```
1
2
3
4
5
```

`mapper.py`

```
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    line = line.strip()
    if line:
        value = int(line)
        print(f"{value}\t1")
```

Salida del mapper

```
1   1
2   1
3   1
4   1
5   1
```

Separador real: tabulador.

Clave = número; valor = 1

Si un número aparece varias veces, se emite un par por cada aparición.

El reducer suma los valores de cada clave

Entrada agrupada por clave; memoria constante

```python
#!/usr/bin/env python3
import sys
current_key = None
current_count = 0
for line in sys.stdin:
    key, count = line.rstrip("\n").split("\t", 1)
    count = int(count)
    if key == current_key:
        current_count += count
    else:
        if current_key is not None:
            print(f"{current_key}\t{current_count}")
        current_key = key
        current_count = count
if current_key is not None:
    print(f"{current_key}\t{current_count}")
```

Acumulación

Mantener la clave actual y su contador.

Cambio de clave

Emitir el grupo anterior e iniciar el siguiente.

Fin de entrada

Emitir el último grupo, si existe.

El mismo reducer sirve para palabras: las claves son texto.

Probar el cálculo localmente

Simular map, agrupación y reduce antes de usar el clúster

```
python3 examples/mapper.py < examples/input.txt \
  | LC_ALL=C sort -k1,1 \
  | python3 examples/reducer.py
```

Salida esperada

```
1   1
2   1
3   1
4   1
5   1
```

Por qué ordenar

El reducer consume claves contiguas. La salida directa de varios mapeadores puede intercalar claves iguales.

Alcance de esta prueba

Comprueba el cálculo con un único flujo. La ejecución distribuida añade particionado, red, planificación y recuperación.

Ejecutar con Hadoop Streaming

Enviar scripts y leer la salida desde HDFS

```
hadoop fs -mkdir -p pmd-hadoop/input
hadoop fs -copyFromLocal examples/input.txt pmd-hadoop/input/numbers.txt
hadoop jar "$HADOOP_HOME"/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files examples/mapper.py,examples/reducer.py \
  -mapper 'python3 mapper.py' \
  -reducer 'python3 reducer.py' \
  -input pmd-hadoop/input/numbers.txt \
  -output pmd-hadoop/output
```

```
hadoop fs -cat 'pmd-hadoop/output/part-*'
```

Antes de enviar el trabajo

El directorio de salida debe ser nuevo y los trabajadores deben tener Python 3.

Programa completo: `examples/run_hadoop.sh`

WordCount: contar palabras

La misma reducción; otra función map

Entrada

```
The fast cat wears no hat.
The cat in the hat ran fast.
```

`word_mapper.py`

```
#!/usr/bin/env python3
import re
import sys
# Preserve case; emit punctuation as separate
    tokens.
for line in sys.stdin:
    for word in re.findall(r"\w+|[^\w\s]", line):
        print(f"{word}\t1")
```

```
def map(dockey, line):
    for word in tokenize(line):
        emit(word, 1)
def reduce(word, values):
    emit(word, sum(values))
```

Pseudocódigo: tokenize separa palabras y signos.

Una política explícita

Se conservan mayúsculas y se separa el punto final: `The` y `the` son claves distintas; `.` también es una clave.

WordCount: agrupación y resultado

Cada clave reúne los valores emitidos por ambos mapeadores

Clave	Valores agrupados	Total
`.`	[1, 1]	2
`The`	[1, 1]	2
`cat`	[1, 1]	2
`fast`	[1, 1]	2
`hat`	[1, 1]	2
`in`	[1]	1
`no`	[1]	1
`ran`	[1]	1
`the`	[1]	1
`wears`	[1]	1

Map

Cada aparición produce `(palabra, 1)`.

Shuffle y sort

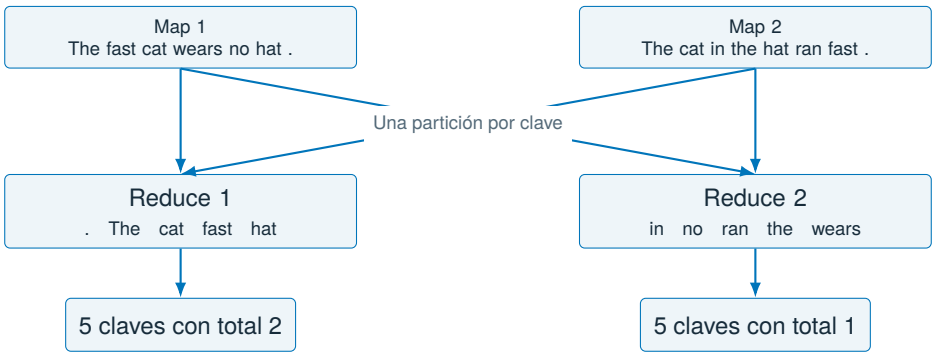
Reunir todas las apariciones de una palabra.

Reduce

Sumar los unos: 15 tokens en total y 10 claves distintas.

WordCount con dos mapeadores y dos reductores

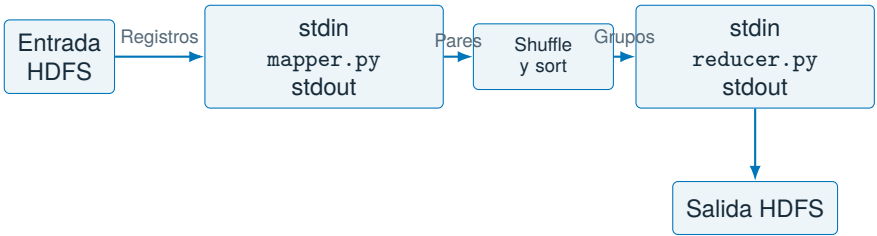
Particionar por clave conecta ambos conjuntos de tareas



Partición ilustrativa. La asignación real depende del particionador; cada clave corresponde a un único reductor.

Hadoop Streaming: entrada y salida estándar

Integrar scripts externos al flujo de MapReduce



Contrato del protocolo

Una línea por registro. En la salida de los scripts, clave y valor se separan mediante un tabulador. Los mensajes de diagnóstico van a `stderr`.

Práctica en Google Colab

Notebooks del curso

Hadoop I

Abrir HadoopI.ipynb

`https://github.com/adigenova/uohpmd/blob/main/code/HadoopI.ipynb`

Hadoop II

Abrir HadoopII.ipynb

`https://github.com/adigenova/uohpmd/blob/main/code/HadoopII.ipynb`

Consultas

Preguntas y comentarios

¿Consultas o comentarios?

Muchas gracias